

Development Of Split Ergonomic Keyboard To Prevent Wrist Fatigue And RSI

*A Product Development Project Report
submitted in partial fulfillment of
the requirement for the B.Tech.
under Biju Patnaik University of Technology, Rourkela.*

Submitted By

ELLA JAYA KISHAN	Regd No. # 2301109389
PAYAL MAHARANA	Regd No. # 2301109407
SUBHAM CHOUDHURY	Regd No. # 2301109431
BIGHNA RAJ BHATTAMISHRA	Regd No. # 2301109379
ADARSH SWARUP MAHARANA	Regd No. # 2421109136
MONALISHA PANDA	Regd No. # 2421109138
MIHIR KUMAR PANDA	Regd No. # 2301109402
MUSKAN PATRA	Regd No. # 2301109403



April - 2026

Under the guidance of

Dr. T. Mita Kumari

**Parala Maharaja Engineering College, Sitalapali
Berhampur, Ganjam, Odisha - 761003**



Estd. – 2009
A Government Engineering College
E-mail – pmecbam@gmail.com
Website – www.pmec.ac.in

PARALA MAHARAJA ENGINEERING COLLEGE

(A constituent college of Biju Patnaik University of Technology, Odisha, Rourkela)
SITALAPALLI: BERHAMPUR: DIST.:-GANJAM: PIN – 761003 (ODISHA)

CERTIFICATE

It is certified that the project work entitled “**Development of a Split Ergonomic Keyboard to prevent Wrist fatigue and RSI**” submitted by “**Ella Jaya Kishan, Payal Maharana, Subham Choudhury, Bighna Raj Bhattamishra, Adarsha Swarup Maharana, Monalisha Panda, Mihir Kumar Panda, Muskan Patra**”, for the partial fulfillment of B. Tech degree is based on her own work carried out under my guidance.

Ella Jaya Kishan (2301109389)

Adarsh Swarup Maharana (2421109136)

Payal Maharana (2301109407)

Monalisha Panda (2421109138)

Subham Choudhury (2301109431)

Mihir Kumar Panda (2301109402)

Bighna Raj Bhattamishra (2301109379)

Muskan Patra (2301109403)

Dr. T. Mita Kumari
Asst. Professor, Dept. of ETC
Parala Maharaja Engineering College
Sitalapalli, Berhampur, Odisha-761003

Dr. Dinesh Kumar Dash
Head of Department
Dept. of ETC, PMEC

ABSTRACT

This project presents the design and development of a split ergonomic concave keyboard aimed at reducing strain associated with Repetitive Strain Injuries (RSI), particularly Carpal Tunnel Syndrome (CTS). The system combines ergonomic mechanical design with embedded firmware to improve typing comfort and long-term usability.

The keyboard consists of two halves, each with 33 keys arranged in a concave layout to support natural finger movement. It uses BLE 5.0-enabled nRF52840 microcontrollers for wireless communication and is powered by individual lithium-ion batteries. The key matrix is configured as 6×6 with 1N4148 diodes for reliable input detection and anti-ghosting, while hot-swappable sockets allow easy maintenance.

The firmware is implemented using the ZMK framework on the Zephyr RTOS, enabling efficient matrix scanning, debouncing, and low-latency Bluetooth communication. A split architecture is used, where one half acts as the central unit and the other as a peripheral, ensuring seamless coordination.

Overall, the project demonstrates a practical and efficient ergonomic keyboard solution integrating hardware design and modern firmware architecture.

ACKNOWLEDGEMENT

I would like to express my immense sense of gratitude to my guide, **Dr. T. Mita Kumari**, for her valuable instructions, guidance and support throughout my seminar.

I again owe my special thanks to **Mr. Nawal Topno**, Product development Coordinator, **Dr. Dinesh Kumar Dash**, Head of the Department for giving me an opportunity to do this report.

And finally thanks to **Prof. Dr. Chittaranjan Mohanty** Principal, PMEC for his continued efforts for enhancing the quality in everything that happens at PMEC. This report is a dedicated contribution towards that greater goal.

ELLA JAYA KISHAN (2301109389)

PAYAL MAHARANA (2301109407)

SUBHAM CHOUDHURY (2301109431)

BIGHNA RAJ BHATTAMISHRA (2301109379)

ADARSH SWARUP MAHARANA (2421109136)

MONALISHA PANDA (2421109138)

MIHIR KUMAR PANDA (2301109402)

MUSKAN PATRA (2301109403)

TABLE OF CONTENTS

ABSTRACT.....	1
ACKNOWLEDGEMENT.....	2
TABLE OF CONTENTS	3
LIST OF FIGURES	5
1. INTRODUCTION.....	1
2. PROBLEM STATEMENT	2
3. OBJECTIVES	3
4. HARDWARE DESIGN	4
4.1 Microcontroller	4
4.2 Mechanical Switches	6
4.3 PCB Design.....	6
4.4 Matrix Configuration	8
4.5 Diodes	9
4.6 Hot-Swap Sockets.....	9
4.7 Wiring Method.....	10
4.8 Power System.....	10
4.9 Mechanical Design.....	11
5. SYSTEM DESIGN.....	12
5.1 Key Press Detection	12
5.2 Debouncing.....	12
5.3 Slave to Master Communication.....	12
5.4 Data Processing and Transmission to Host.....	13
5.5 Power Management	13
6. IMPLEMENTATION PROCESS	14
6.1 Component Procurement	14
6.2 CAD Design and 3D Printing	14
6.3 PCB Preparation.....	15
6.4 Matrix Wiring	15
6.5 Microcontroller Integration.....	15
6.6 Firmware Flashing	16
6.7 Enclosure Assembly.....	16
7. FIRMWARE ARCHITECTURE.....	17
7.1 The Zephyr Real-Time Operating System	17
7.1.1 What is an RTOS and Why Does a Keyboard Need One?	17
7.1.2 Preemptive Multitasking in Zephyr	17
7.1.3 Zephyr's Device Tree and Hardware Abstraction.....	18
7.2 ZMK Firmware Framework Architecture.....	18

7.2.1	The Shield System	18
7.2.2	The Kconfig Build System.....	19
7.3	CI/CD Pipeline via GitHub Actions	19
7.3.1	The build.yaml Configuration File	20
7.3.2	Zephyr West Build System.....	20
7.3.3	Advantages of Cloud-Based Compilation	21
7.4	Hardware Definition: The Device Tree Overlay File	21
7.4.1	GPIO Bank Architecture on the nRF52840	21
7.4.2	Annotated Overlay Structure	21
7.4.3	The diode-direction Property.....	22
7.5	Keymap Architecture: The .keymap File	23
7.5.1	Layered Logic: Layered System	23
7.5.2	ZMK Behaviours: Atomic Key Actions	24
7.5.3	Combos: Multi-Key Simultaneous Triggers.....	25
7.6	Split Keyboard BLE Communication Protocol	26
7.6.1	BLE Central and Peripheral Roles	26
7.6.2	The 7.5 ms Connection Interval.....	27
7.6.3	ZMK Split Transport Data Format	27
7.6.4	BLE Bonding and Security	27
8.	SYSTEM WORKING AND VALIDATION.....	29
8.1	Signal Debouncing Algorithm.....	29
8.1.1	The Eager Debounce Algorithm	29
8.2	Power Management and Sleep State Machine	30
8.2.1	Active Mode	30
8.2.2	Idle Mode	30
8.2.3	Deep Sleep Mode	31
8.2.4	Power Consumption Analysis and Battery Life Estimation	31
8.3	Testing and Results	31
8.3.1	BLE Pairing and Connection Reliability	32
8.3.2	Key Registration Accuracy	32
8.3.3	Typing Ergonomics Evaluation	33
8.3.4	Battery Longevity Test	33
9.	CONCLUSION	34
10.	FUTURE SCOPE.....	35
	REFERENCES.....	36

LIST OF FIGURES

Figure 4.1: Pro Micro nRF52840 Pinout Diagram	[Page 5]
Figure 4.2: Mechanical Key Switch Design	[Page 6]
Figure 4.3: PCB Design	[Page 7]
Figure 4.4: Keyboard Matrix Configuration	[Page 8]
Figure 4.5: 1N4148 Signal Diode	[Page 9]
Figure 4.6: Hot swappable sockets.....	[Page 9]
Figure 4.9: Mechanical Shell design.....	[Page 11]

1. INTRODUCTION

Computers are an essential part of modern professional life, requiring long hours of continuous typing. However, prolonged use of conventional flat keyboards leads to physical strain, particularly Carpal Tunnel Syndrome (CTS), caused by sustained pressure on the wrist's median nerve. Standard keyboard designs force unnatural wrist positions, such as ulnar deviation, resulting in discomfort, fatigue, and long-term injury. Although ergonomic keyboards exist, many are expensive, lack customization, or fail to fully address these biomechanical issues. This project focuses on the design and development of a split ergonomic concave keyboard to improve typing comfort and reduce strain.

The keyboard consists of two independent halves that can be positioned freely, allowing a natural wrist posture. Each half features a concave key layout aligned with the natural arc of finger movement, reducing effort during typing. Wireless Bluetooth communication eliminates the need for cables between the halves.

This report presents the complete hardware design and implementation process, including component selection, circuit design, assembly, and testing, providing a practical ergonomic solution for long-duration keyboard use.

1.1 Background and Motivation

Many professionals experience wrist pain and fatigue due to prolonged keyboard use, often linked to CTS. Traditional keyboard designs do not align with natural hand movement and posture, increasing physical strain over time. This project is motivated by the need to develop a more ergonomic and accessible keyboard design.

1.2 Scope of the Project

This project focuses on the design and implementation of a split ergonomic keyboard. It includes component selection, circuit design, wiring, power system, and 3D printed enclosure. Firmware aspects are mentioned only where necessary. The final system functions as a standard Bluetooth HID keyboard compatible with common devices.

2. PROBLEM STATEMENT

The primary problem addressed in this project is the physical strain and long-term injury caused by conventional flat keyboard designs. Despite widespread use, these keyboards do not align with human ergonomic principles and continue to impose harmful postures during prolonged typing.

2.1 Unnatural Wrist Positioning

Conventional keyboards force the wrists inward (ulnar deviation), creating continuous stress on tendons and the median nerve. Over time, this leads to conditions such as Carpal Tunnel Syndrome (CTS), causing pain, numbness, and reduced hand function.

2.2 Mismatch with Natural Finger Movement

Human fingers move in a natural arc, whereas flat keyboards require linear movement. This mismatch increases effort per keystroke and leads to cumulative fatigue during extended use.

2.3 Lack of Adjustability

Standard keyboards have a fixed layout that does not accommodate differences in shoulder width or hand positioning. This forces users into unnatural postures, increasing strain over time.

2.4 High Cost of Ergonomic Alternatives

Existing ergonomic keyboards address some of these issues but are often expensive and inaccessible. This creates a need for a cost-effective and customizable ergonomic solution.

3. OBJECTIVES

The primary objectives of this project are as follows:

1. To design and implement a split ergonomic keyboard that allows independent positioning of both halves to achieve a natural and comfortable typing posture.
2. To incorporate a concave key-well geometry that aligns with the natural motion of fingers, reducing typing effort and strain.
3. To develop a reliable hardware system by integrating appropriate components, including a BLE-enabled microcontroller, mechanical switches, diodes, hot-swappable sockets, and a battery system.
4. To design an efficient key matrix with anti-ghosting support for accurate detection of multiple simultaneous keypresses.
5. To implement a firmware system using the ZMK framework that supports matrix scanning, key debouncing, layered key mapping, and low-latency input processing.
6. To establish wireless communication using Bluetooth Low Energy (BLE) for both split-half interaction and host connectivity, eliminating the need for physical interconnections.
7. To design and fabricate a custom 3D-printed enclosure that supports ergonomic structure while securely housing all components.
8. To validate the complete system through testing, ensuring reliable key registration, stable wireless performance, effective power management, and improved ergonomic comfort.

4. HARDWARE DESIGN

The hardware design of the split ergonomic keyboard involves the selection and integration of several electronic and mechanical components. Each component was chosen for specific reasons related to functionality, reliability, ergonomic, and practicality. The following subsections describe each hardware component in detail.

4.1 Microcontroller

The brain of each keyboard half is the Pro Micro nRF52840 development board. This small but powerful microcontroller board is built around the Nordic Semiconductor nRF52840 System-on-Chip (SoC). It was chosen for this project for several key reasons.

The nRF52840 features a 64 MHz ARM Cortex-M4 processor, which is more than powerful enough to scan the key matrix, process keypresses, run the Bluetooth stack, and manage power all simultaneously. Most importantly for this project, the chip includes a native Bluetooth 5.0 radio, meaning no additional Bluetooth module is needed. Both halves of the keyboard use this same chip, and the wireless communication between them is handled entirely through the built-in BLE radio.

The chip operates at 3.3 V and can run directly from a Lithium-ion battery (which provides 3.0–4.2 V). The Pro Micro nRF52840 board includes an onboard USB-C charging circuit, allowing the battery to be recharged conveniently without removing it from the keyboard. Although the nRF52840 microcontroller supports up to 48 GPIO pins internally, only a limited subset of 29 pins are exposed on the Pro Micro form factor. Out of these, 12 GPIO pins are used for the 6×6 key matrix, leaving additional pins available for other functions.

The firmware running on both boards is based on the open-source ZMK keyboard firmware framework, which provides built-in support for the nRF52840, Bluetooth HID communication, and split keyboard functionality. One board is configured as the master (BLE central + HID peripheral to the host) and the other as the slave (BLE peripheral sending key events to master).

PINOUT



Charge current selection jumper:
Connect this jumper "BOOST", the charging current increases from 100ma to 300ma. Only do this if your battery is larger than 500mAh to avoid explosion.

External VCC cutoff control:
When P0.13 is set low, the power supply to the 3.3V ~VCC pin will be turned off. This is useful for components that use less power when idle, such as RGB LEDs.

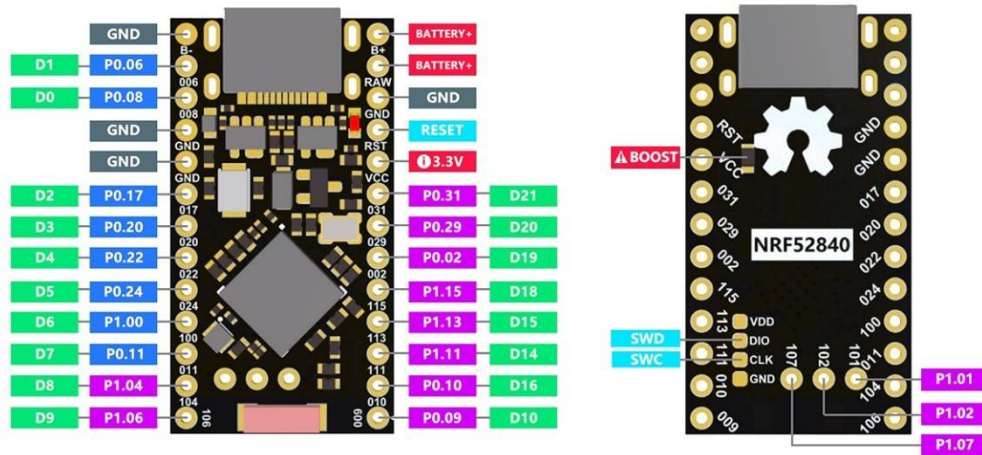


Figure 4.1: Pro Micro nRF52840 Pinout Diagram

Parameter	Specification
Processor	ARM Cortex-M4 @ 64 MHz
Flash Memory	1 MB
RAM	256 KB
Wireless	Bluetooth 5.0 / BLE (integrated)
Operating Voltage	3.3V
GPIO Pins	48 (P0 + P1 banks)
Charging	USB-C onboard LiPo charge circuit

Table 4.1: Pro Micro nRF52840 Specifications

4.2 Mechanical Switches

The mechanical key switches are the mechanical components that register a keypress. When a key is pressed, the stem moves downward, compressing the spring and bringing the gold crosspoint contacts together to complete the circuit.

For this project, Outemu Brown mechanical switches were selected. The Brown type is classified as a tactile switch, meaning it provides a small, noticeable physical bump when the key reaches its actuation point. This tactile feedback is important for ergonomic typing: it allows the typist to feel when the key has registered without having to press it all the way down to the bottom. This reduces the impact force on the fingertips and tendons with each keystroke, directly contributing to reduced fatigue and strain.

Key specifications of the Outemu Brown switch include an actuation force of 45 gf (gram-force), a pre-travel distance of 2.0 mm to actuation, and a total travel of 4.0 mm. The switch has a rated lifespan of 50 million keystrokes, which translates to many years of normal use. A total of 66 switches are used in the project (33 per half), one for each key position.

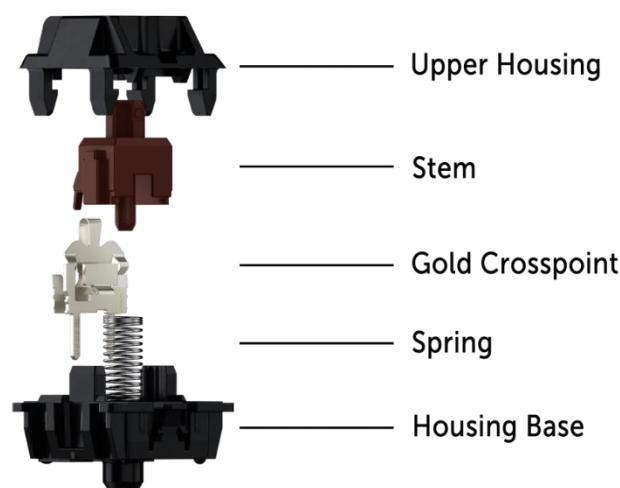


Figure 4.2: Mechanical Key Switch Design

4.3 PCB Design

A printed circuit board (PCB) is used in each keyboard half to provide a structured mounting surface for the key switches and to create the electrical connections required for the matrix circuit. Rather than using a single large PCB for the entire

keyboard half, this design uses a per-key PCB approach, where small individual PCBs support the switch and diode at each key position. These are connected by the hand-wired matrix described in Section 4.7.

Each PCB has three holes: two holes accept the electrical pins from the Gateron hot-swap socket (which connects to the key switch), and one hole provides additional mechanical support to prevent the socket from rotating or lifting away from the PCB under repeated use. The PCBs are arranged across the concave key-well surface of the 3D printed enclosure, mounted at the appropriate angles to match the curvature of the key-well.



Figure 4.3: PCB Design

The per-key PCB approach was chosen instead of a single integrated PCB for practical reasons. A single PCB for the entire concave key-well would need to be flexible (using expensive flex PCB materials) or consist of multiple rigid segments joined together. The per-key PCB approach allows each key to be mounted at a slightly different angle to match the concave surface geometry, while keeping the PCB design simple, inexpensive, and manufacturable through standard PCB fabrication services.

4.4 Matrix Configuration

Connecting 33 individual key switches each with their own dedicated microcontroller pin would require 33 GPIO pins per half far more than practical. Instead, the switches are arranged in a matrix configuration, which allows many more switches to be addressed using far fewer GPIO pins.

In a matrix design, the switches are arranged in a grid of rows and columns. Each switch sits at the intersection of one row line and one column line. To detect which key is pressed, the microcontroller activates one row at a time by sending a signal down that row. It then checks all the column lines. If a particular column shows a signal, it means the switch at the intersection of that row and column is pressed. The microcontroller repeats this process for each row very rapidly (1,000 times per second), so it appears to detect all key presses simultaneously.

This keyboard uses a 6×6 matrix, providing 36 addressable positions. Since 33 keys are physically present per half, three positions in the matrix are left unused. The 6×6 configuration requires only 6 GPIO pins for rows and 6 GPIO pins for columns, a total of 12 pins compared to the 33 pins that direct wiring would require. This greatly simplifies the wiring and reduces the demands on the GPIO resources.

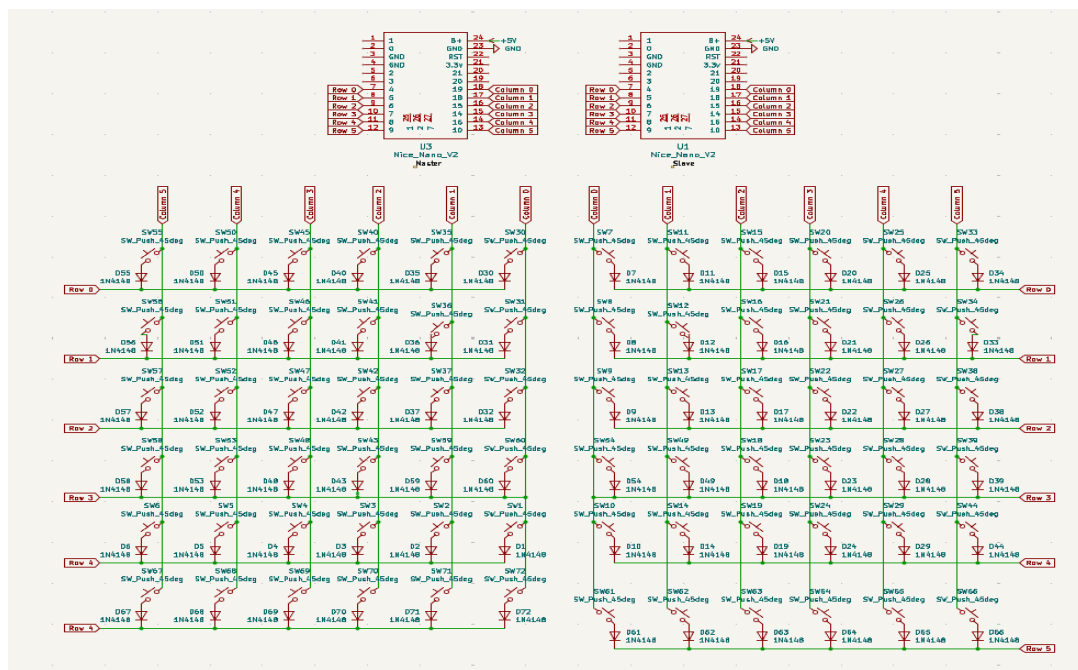


Figure 4.4: Keyboard Matrix configuration

4.5 Diodes

In a key matrix, multiple simultaneous key presses can cause ghosting, where unintended signals lead to incorrect key detection. To prevent this, a 1N4148 diode is placed in series with each switch.

The diode allows current to flow in only one direction, ensuring signals pass correctly from row to column and eliminating false key detection. This enables reliable multi-key input (N-key rollover).



Figure 4.5: 1N4148 signal diode

The 1N4148 diode was selected due to its fast switching speed, low cost, and compact size. A total of 66 diodes are used (one per switch).

4.6 Hot-Swap Sockets

Hot-swap sockets allow mechanical switches to be installed and removed without soldering. The sockets are soldered to the PCB, and the switch pins are inserted directly into them.

They are compatible with standard MX-style switches and provide reliable electrical contact with minimal resistance. This enables easy switch replacement and improves maintainability of the keyboard.



Figure 4.6: Hot Swappable Sockets

4.7 Wiring Method

The electrical connections forming the key matrix are implemented using hand wiring rather than a single integrated PCB. In this approach, individual insulated copper wires are manually soldered to each switch to establish the row and column connections required for matrix scanning.

Hand wiring was chosen over a custom PCB for several practical reasons. The concave three-dimensional geometry of the key-well makes it difficult to design a rigid flat PCB that conforms to the surface. While flexible PCBs are a possible alternative, they introduce additional cost and manufacturing complexity. Hand wiring allows each key switch to be positioned at varying angles to match the concave layout, with wires routed flexibly between them. Additionally, for a prototype-scale implementation, hand wiring avoids the time and expense associated with custom PCB fabrication.

Small insulated copper wires are used for all matrix connections. These wires are thin enough to be routed neatly between closely spaced switches while remaining sufficiently durable for assembly and handling. To improve clarity during assembly and debugging, row and column connections are distinguished using different wire colors.

4.8 Power System

Each keyboard half is powered by its own independent Lithium-ion (Li-ion) rechargeable battery in a Lithium-Polymer (LiPo) pouch cell package. The nominal voltage of the cell is 3.7 V, which is directly compatible with the Pro Micro nRF52840 board's operating voltage range (1.7 V to 5.5 V). The battery capacity is 500 mAh, which provides adequate runtime for normal use between charges.

The Pro Micro nRF52840 board includes a built-in battery charging circuit accessible via its USB-C port. When connected to any USB charger or computer via USB-C, the board automatically charges the battery using a standard constant-current / constant-voltage (CC/CV) charging profile, which is the safe and correct method for

Li-ion cells. The firmware makes use of the nRF52840's built-in power management features including low-power sleep modes that activate after periods of inactivity to extend the battery life between charges.

Under typical use conditions (active typing for several hours per day, with the keyboard entering sleep mode when not in use), the 500 mAh battery per half provides a runtime of several weeks between charges. The two halves charge independently via their own USB-C ports. There is no need to carry or manage any external battery pack.

4.9 Mechanical Design

The keyboard enclosure was designed using CAD software and fabricated using FDM 3D printing with PLA material. The design consists of two split halves, each shaped to support ergonomic hand positioning.

A key feature is the concave key-well, which aligns with the natural arc of finger movement, reducing reach and improving typing comfort. The left and right halves are mirror images and can be positioned independently.

Each enclosure includes mounting provisions for PCBs, microcontroller, and battery, along with a USB-C cutout for charging. Components are secured using M2 standoffs and screws, allowing easy assembly and maintenance.

PLA was selected for its ease of printing, sufficient strength, and dimensional accuracy. The enclosure was printed with a 0.15 mm layer height and 40% infill to achieve a balance between strength and print time.

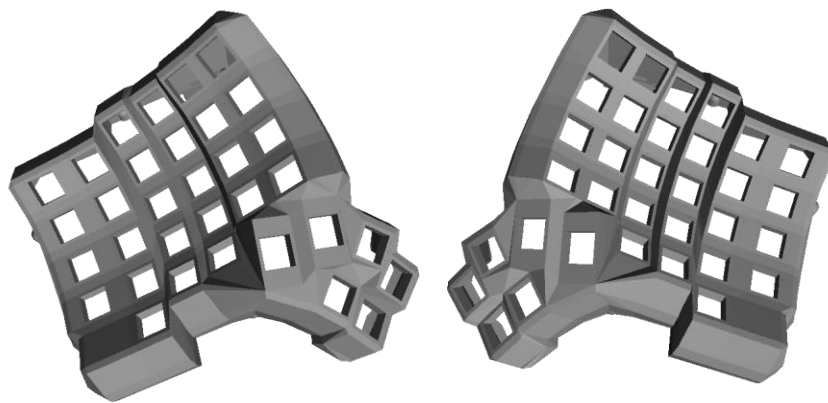


Figure 4.9: Mechanical Shell design

5. SYSTEM DESIGN

The operation of the split ergonomic keyboard can be understood by tracing the sequence of events from a key press to the final output on the host device. The system integrates matrix scanning, signal processing, wireless communication, and power management to ensure reliable performance.

5.1 Key Press Detection

Each keyboard half contains a 6×6 key matrix, which is continuously scanned by the nRF52840 microcontroller at a rate of approximately 1 kHz. The scanning process is performed by sequentially activating each row line while monitoring the column lines.

When a key is pressed, the switch closes the circuit between its corresponding row and column. During the active row cycle, the microcontroller detects a signal on the associated column line, allowing it to identify the exact position of the key within the matrix.

The inclusion of 1N4148 diodes at each switch ensures that current flows in only one direction. This prevents unintended signal paths and enables accurate detection of multiple simultaneous key presses without ghosting.

Once detected, the key position is mapped to a predefined key function using a keymap table stored in firmware.

5.2 Debouncing

Mechanical switches exhibit contact bounce, where the electrical signal fluctuates briefly when a key is pressed or released. Without correction, this would result in multiple unintended key registrations.

To address this, a debounce mechanism is implemented in firmware. The microcontroller confirms a keypress only after the signal remains stable for a short duration (typically around 5 ms). This ensures that each physical key press is registered as a single input event.

5.3 Slave to Master Communication

When the slave half (left keyboard) detects a keypress or key release, it encodes the current state of its full 6×6 matrix into a compact Bluetooth packet and sends it to the master half (right keyboard) over the BLE link between the two halves. This transmission happens within one BLE connection interval, which is set to 7.5 milliseconds fast enough that the delay is imperceptible to the user.

5.4 Data Processing and Transmission to Host

The master half receives input from both its own matrix and the slave half. It combines these inputs and generates a standard Human Interface Device (HID) report.

This report is transmitted to the host computer via BLE. The host system interprets the HID report and executes the corresponding action, such as displaying characters or triggering shortcuts. From the host's perspective, the keyboard behaves as a single unified input device.

5.5 Power Management

To optimize battery life, the system incorporates multiple power-saving strategies. During active use, the microcontroller operates at full scanning speed and maintains frequent BLE communication.

When idle, the system reduces the scanning frequency and increases BLE connection intervals to minimize power consumption. After extended inactivity, the microcontroller enters a low-power sleep mode.

Any key press generates an interrupt signal that wakes the system instantly, restoring normal operation without noticeable delay. This approach ensures efficient power usage while maintaining responsiveness.\

6. IMPLEMENTATION PROCESS

The implementation of the split ergonomic keyboard followed a structured sequence of steps, from initial design and component procurement through to final assembly and testing. The following subsections describe each phase of the implementation.

6.1 Component Procurement

All components were sourced from electronic component distributors and online marketplaces. The Pro Micro nRF52840 boards were ordered from an online market place. Outemu Brown switches (66 pieces), Gateron hot-swap sockets (66 pieces), and 1N4148 diodes (66 pieces) were ordered in single quantities sufficient for the full keyboard build. 28 AWG insulated wire (in multiple colours), M2 brass standoffs, and M2 stainless steel screws were sourced from general electronics suppliers. PLA filament for 3D printing was already available in the project lab.

6.2 CAD Design and 3D Printing

The enclosure for each keyboard half was designed in FreeCAD, an open-source parametric CAD application. The design process began by measuring the dimensions of all internal components (switches, sockets, PCBs, battery, microcontroller board) and creating 3D models of each. These models were assembled in the CAD software to verify spatial compatibility before the enclosure was designed around them.

The concave key-well geometry was derived from measurements of the natural resting position of the hand. The key positions were distributed along a cylindrical arc with a radius of approximately 55 mm, matching the average arc of finger movement during typing. The completed CAD designs were exported as STL files (one for each half) and sliced for 3D printing using PrusaSlicer software. The print settings used were: layer height 0.15 mm, infill 40%, support structures for the battery compartment undercuts, and PLA filament at 205°C print temperature and 60°C bed temperature. Each half required approximately 8–10 hours of print time.

6.3 PCB Preparation

The per-key PCBs were ordered from a PCB fabrication service using custom Gerber files generated from the PCB design. Each small PCB has the three-hole layout. Upon receipt, the Gateron hot-swap sockets were soldered to the PCBs manually using a standard soldering iron. The socket pins fit precisely into the two electrical holes, and the body pin fits into the mechanical retention hole. All solder joints were inspected visually and tested for continuity using a digital multimeter.

6.4 Matrix Wiring

After PCB preparation, the hand-wired matrix was assembled. This is the most time-intensive step of the build. Insulated copper wires were cut to appropriate lengths and stripped at both ends. Row wires were connected horizontally across all switch positions in each row, soldered to the anode (positive) side of the 1N4148 diode at each node. Column wires were connected vertically across switch positions in each column, soldered to the column pin of the hot-swap socket at each node.

The cathode of each diode was soldered to the socket pin, completing the series circuit: row wire → diode anode → diode cathode → socket pin → column wire. All 33 diodes and all 12 row/column wires per half were soldered in this manner. After wiring, each matrix connection was tested with a multimeter in continuity mode to confirm correct connections and identify any accidental short circuits or missed solder joints.

6.5 Microcontroller Integration

The row wires and column wires were connected to the designated GPIO pins of the Pro Micro nRF52840 board. The specific pin assignments were defined in the ZMK firmware configuration file. The battery positive wire was connected to the board's battery positive input, and the battery negative wire to the ground. The board was then mounted in the enclosure using standoffs and screws, with the USB-C port aligned to the enclosure's charging port opening.

6.6 Firmware Flashing

The ZMK firmware was configured with the keymap (the assignment of keyboard functions to each matrix position), pin assignments, debounce settings, BLE connection parameters, and power management settings for both the master and slave halves. The compiled firmware binary was flashed to each board via the USB-C connection using the board's built-in UF2 bootloader a simple drag-and-drop process that requires no special programming hardware. After flashing, the boards were powered and the BLE pairing between master and slave was established.

6.7 Enclosure Assembly

The completed electronic assembly PCBs, wiring, microcontroller board, and battery was carefully fitted into the 3D printed enclosure. The PCBs were attached to the printed mounting bosses using M2 standoffs, the battery was placed in its compartment and secured with a foam pad, and all wires were routed and secured. Key switches were then inserted into the Gateron hot-swap sockets (no soldering required). PBT keycaps were fitted onto the switch stems. The enclosure lid was then attached using M2 screws, completing the physical assembly of each half.

7. FIRMWARE ARCHITECTURE

The firmware layer of the Split Ergonomic Concave Keyboard is as critical to its performance as the hardware it runs upon. Unlike conventional embedded software that runs as a single infinite loop, the firmware for this keyboard is built upon a sophisticated, multi-threaded real-time operating system. This chapter provides a comprehensive technical analysis of the ZMK firmware framework, the Zephyr RTOS upon which it operates, the automated cloud-based compilation pipeline, a line-by-line examination of the hardware definition and keymap files, and a detailed description of the split-half Bluetooth Low Energy communication protocol.

7.1 The Zephyr Real-Time Operating System

ZMK Firmware does not execute on bare metal. It is built as an application running on top of the Zephyr Project RTOS an open-source, scalable, real-time operating system hosted by the Linux Foundation. This architectural decision is fundamental to the keyboard's ability to handle multiple concurrent tasks reliably.

7.1.1 What is an RTOS and Why Does a Keyboard Need One?

A Real-Time Operating System (RTOS) is a kernel designed to service tasks within guaranteed time bounds. Unlike general-purpose operating systems (e.g., Linux), an RTOS prioritises determinism the guarantee that a high-priority event will be serviced within a known, bounded latency. For a keyboard, this is essential: a user expects every keypress to register instantaneously. A bare-metal loop-based approach risks 'missing' a keypress if, for example, the processor is occupied processing a Bluetooth packet. An RTOS eliminates this risk through preemptive scheduling.

7.1.2 Preemptive Multitasking in Zephyr

Zephyr implements a preemptive, priority-based thread scheduler. Each concurrent activity in ZMK runs as a separate kernel thread with an assigned priority level. When a higher-priority thread becomes ready to execute (for example, an interrupt indicates a key has been pressed), the Zephyr scheduler immediately suspends the currently running lower-priority thread and switches context to the higher-priority one. This context switch is extremely fast on the ARM Cortex-M4, taking fewer than 100 CPU clock cycles.

The primary concurrent workloads in ZMK on this keyboard are as follows. The Matrix Scanning Thread polls the 6x6 GPIO matrix at a high rate (approximately every 1 ms) to detect key state changes. This thread is assigned a relatively high priority to ensure key events are never missed. The BLE Host Stack (provided by Zephyr's Bluetooth subsystem) manages all Bluetooth communication: advertising, connection management, security pairing, and HID report transmission. The Split Transport Thread handles the serialisation and transmission of key events from the peripheral half to the central half over BLE. The Power Management Thread monitors activity and transitions the system between active, idle, and deep-sleep power states.

7.1.3 Zephyr's Device Tree and Hardware Abstraction

A key feature inherited from Zephyr is its use of a Device Tree (DTS) for hardware description. Borrowed from the Linux kernel, the Device Tree is a data structure that describes the hardware topology to the operating system. Rather than hard-coding hardware addresses and GPIO pin numbers directly into application code, they are declared in a `.overlay`` file that is compiled alongside the firmware. This creates a clean separation between hardware description and application logic, making the firmware portable and maintainable. When ZMK compiles, the Zephyr build system (CMake/west) merges the base board Device Tree (for the nice!nano v2) with the shield overlay (for the Dactyl Manuform) to produce a complete hardware description.

7.2 ZMK Firmware Framework Architecture

ZMK (Zephyr Mechanical Keyboard) is an open-source keyboard firmware project built natively on Zephyr RTOS. It was designed from the ground up to support wireless keyboards using modern BLE-capable SoCs such as the Nordic nRF52840. ZMK's architecture is modular, separating concerns into distinct components: the hardware abstraction layer (Device Tree overlays), the behaviour engine (keymap files), the transport layer (split keyboard protocol), and the HID output layer (USB and BLE HID).

7.2.1 The Shield System

In ZMK, a 'shield' is the firmware representation of a keyboard PCB or handwired build. It is analogous to a 'keyboard' definition in QMK firmware. The shield for this project is named ``dactyl_manuform`` and resides under the ``boards/shields/dactyl_manuform/`` directory in the repository. A shield consists of at

minimum three files: a `.overlay` file (Device Tree overlay for GPIO pin assignments), a `.keymap` file (key layout and behaviour definitions), and a `Kconfig.shield` file (compile-time feature flags). Additional `.conf` files may be present to set Kconfig options specific to each half (e.g., enabling experimental BLE features on Linux hosts).

7.2.2 The Kconfig Build System

ZMK uses Zephyr's Kconfig system derived from the Linux kernel to manage compile-time configuration. Kconfig options are boolean or integer values that enable or disable entire subsystems before compilation. For this keyboard, key Kconfig options include `CONFIG_ZMK_SPLIT=y` (enables split keyboard support), `CONFIG_ZMK_BLE=y` (enables Bluetooth Low Energy output), `CONFIG_ZMK_USB=n` (disables USB HID since the keyboard is wireless), and `CONFIG_ZMK_SLEEP=y` (enables the power management sleep subsystem). These options are set in the `.conf` files within the `config/` directory and in the shield's `Kconfig.shield` file.

7.3 CI/CD Pipeline via GitHub Actions

One of the most significant practical advantages of ZMK over legacy firmware frameworks such as QMK is its fully cloud-based compilation pipeline. Building ZMK locally requires installing the Zephyr SDK (a multi-gigabyte ARM cross-compilation toolchain), the `west` meta-tool, Python dependencies, and correctly configuring environment variables, a process that can take several hours and is prone to version conflicts. ZMK eliminates this barrier entirely through GitHub Actions, a cloud-based CI/CD platform integrated directly into GitHub repositories.

7.3.1 The `build.yaml` Configuration File

The compilation targets are defined in the `build.yaml` file at the root of the repository. For this project, the file specifies the following build matrix:

```
# build.yaml ZMK firmware build matrix for tuxy-streamer/zmk-config
---
include:
- board: nice_nano_v2
  shield: dactyl_manuform_left
- board: nice_nano_v2
  shield: dactyl_manuform_right
```

This YAML file instructs the GitHub Actions workflow to compile two firmware binaries: one for the left half (shield: ``dactyl_manuform_left``) and one for the right half (shield: ``dactyl_manuform_right``), both targeting the ``nice_nano_v2`` board. The board identifier ``nice_nano_v2`` corresponds to the nice!nano v2 which is functionally identical to the Pro Micro nRF52840 in terms of the nRF52840 SoC and pin mapping used in this build.

7.3.2 Zephyr West Build System

The Zephyr RTOS employs **West** as a meta-tool to manage its multi-repository structure and build process. West is a command-line utility that simplifies dependency management, source synchronization, and firmware compilation within Zephyr-based projects.

In a typical Zephyr environment, the firmware is composed of multiple components, including the core RTOS, hardware abstraction layers, and application-specific modules. West uses a manifest file ([west.yml](#)) to define and manage these dependencies, ensuring consistency across the development environment.

West integrates with the CMake build system to configure and compile the firmware for the target hardware. It also supports additional functionalities such as updating project repositories and flashing compiled binaries to the microcontroller.

In this project, West is utilized as part of the ZMK firmware build system. Although firmware compilation is primarily performed through a cloud-based CI/CD pipeline using GitHub Actions, West operates in the background to fetch dependencies and execute the build process. This ensures a reproducible and streamlined workflow without requiring complex local setup.

7.3.3 Advantages of Cloud-Based Compilation

Aspect	Local Toolchain	GitHub Actions (Cloud)
Setup Time	2–4 hours (SDK install, PATH config)	Zero no local install required
Disk Space	~10 GB (Zephyr SDK + dependencies)	Consumed on cloud runner only
Reproducibility	Risk of version mismatch	Pinned SDK version per workflow
Accessibility	Requires Linux/macOS expertise	Edit YAML in browser; build online
Version Control	Manual binary management	Artifacts auto-attached to each commit
Build Triggers	Manual command required	Automatic on every git push

Table 7.1: Comparison of Local Toolchain vs. Cloud-Based Firmware Compilation

7.4 Hardware Definition: The Device Tree Overlay File

The `.overlay` file is the firmware's hardware description. It uses the Device Tree Source (DTS) syntax, a structured hierarchical format originally developed for the Linux kernel. For the Dactyl Manuform shield, the overlay tells ZMK precisely which physical GPIO pins on the nRF52840 correspond to which rows and columns of the key matrix.

7.4.1 GPIO Bank Architecture on the nRF52840

The nRF52840 organises its 48 GPIO pins into two banks. GPIO Bank 0 (`&gpio0`) contains pins P0.00 through P0.31, and GPIO Bank 1 (`&gpio1`) contains pins P1.00 through P1.15. On the Pro Micro / nice!nano v2 form factor, the silkscreen labels (e.g., '2', '3', 'A0', 'SCK') do not directly correspond to nRF52840 pin numbers. The mapping from silkscreen label to actual nRF52840 pin must be looked up in the board's pinout diagram. An important rule for the Device Tree is: for pins with a number greater than 31 (i.e., P1.xx pins), the pin number used in the overlay is the value on the silkscreen minus 32, and the bank changes from `&gpio0` to `&gpio1`.

7.4.2 Annotated Overlay Structure

The overlay file for the left half of the Dactyl Manuform follows the standard ZMK matrix scanning driver pattern. The structure declares the key matrix as a DTS

node with child nodes for rows (outputs driven HIGH/LOW by the MCU) and columns (inputs read by the MCU). A representative overlay for a 6-row, 6-column half is structured as follows:

```
/*
 * dactyl_manuform_left.overlay
 * Maps physical nRF52840 GPIO pins to the logical 6x6 key matrix.
 */

/ {
    chosen {
        zmk,kscan = &kscan0; /* Declares the key-scan driver to use */
    };

    kscan0: kscan_0 {
        compatible = "zmk,kscan-gpio-matrix";
        label = "KSCAN";
        diode-direction = "row2col";
        /* row2col: current flows FROM row pin THROUGH diode TO column pin.
         * Matches the 1N4148 orientation used in hardware assembly. */

        row-gpios
            = <&gpio0 31 GPIO_ACTIVE_HIGH> /* Row 0: P0.31 */
            , <&gpio0 29 GPIO_ACTIVE_HIGH> /* Row 1: P0.29 */
            , <&gpio0 2 GPIO_ACTIVE_HIGH> /* Row 2: P0.02 */
            , <&gpio1 15 GPIO_ACTIVE_HIGH> /* Row 3: P1.15 */
            , <&gpio1 13 GPIO_ACTIVE_HIGH> /* Row 4: P1.13 */
            , <&gpio1 11 GPIO_ACTIVE_HIGH> /* Row 5: P1.11 */
            ;

        col-gpios
            = <&gpio0 22 (GPIO_ACTIVE_HIGH | GPIO_PULL_DOWN)> /* Col 0: P0.22 */
            , <&gpio0 24 (GPIO_ACTIVE_HIGH | GPIO_PULL_DOWN)> /* Col 1: P0.24 */
            , <&gpio1 0 (GPIO_ACTIVE_HIGH | GPIO_PULL_DOWN)> /* Col 2: P1.00 */
            , <&gpio0 11 (GPIO_ACTIVE_HIGH | GPIO_PULL_DOWN)> /* Col 3: P0.11 */
            , <&gpio1 4 (GPIO_ACTIVE_HIGH | GPIO_PULL_DOWN)> /* Col 4: P1.04 */
            , <&gpio1 6 (GPIO_ACTIVE_HIGH | GPIO_PULL_DOWN)> /* Col 5: P1.06 */
            ;
    };
};
```

Each line within `row-gpios` and `col-gpios` is a reference to a specific GPIO pin. The first argument is the GPIO bank node (`&gpio0` or `&gpio1`), the second is the pin number within that bank, and the third is a flag macro. For row pins, `GPIO\_ACTIVE\_HIGH` means the ZMK matrix scanner will drive the pin to logical HIGH (3.3 V) when activating that row for scanning. For column pins, `GPIO\_ACTIVE\_HIGH | GPIO\_PULL\_DOWN` means the pin reads the input and uses the nRF52840's internal pull-down resistor to ensure the pin reads LOW when no key is pressed, preventing false triggers from floating inputs.

7.4.3 The diode-direction Property

The ``diode-direction = "row2col"`` property is critical. It tells the ZMK matrix scanning driver the direction in which current flows through the 1N4148 diodes. In the hardware assembly, the diode anode is connected to the row wire and the cathode to the column wire. This means when a row is driven HIGH and a switch is pressed, current flows through the diode and switch to the column pin. Setting ``row2col`` correctly matches the firmware scanning logic to the physical diode orientation. An incorrect diode direction setting would cause the matrix scanner to drive and read the wrong pins, resulting in no keys being detected.

7.5 Keymap Architecture: The .keymap File

The `.keymap`` file is the core user-configurable component of ZMK. Written in the same DTS-like syntax as the overlay file, it defines the complete logical behaviour of the keyboard: what each key does, how layers interact, what combinations of keys trigger special actions, and how modifier keys behave. Understanding this file requires understanding three key ZMK concepts: Layers, Combos, and Behaviours.

7.5.1 Layered Logic: Layered System

ZMK implements a layer system similar to QMK. A layer is a complete map of key bindings that overlays the default map when activated. This allows a single physical keyboard to produce far more than 66 unique keycodes by switching between logical layers.

The Dactyl Manuform keymap for this project implements three primary layers. The Base Layer (Layer 0) contains the standard QWERTY alphanumeric layout, modifier keys (Shift, Ctrl, Alt, GUI/Super), and the thumb-cluster keys (Space, Enter, Backspace, Tab, Escape, and the layer-momentary keys). This layer is active by default at all times. The Lower Layer (Layer 1) is activated by holding the left thumb's lower key and contains numeric keys (0–9) mapped to the top row, navigation keys (arrow cluster), and common symbols (&, *, (,), etc.). The Raise Layer (Layer 2) is activated by holding the right thumb's raise key and contains function keys (F1–F12), media control keys (volume, play/pause), and additional symbols (!, @, #, \$, etc.).

The keymap file structure for these three layers follows this pattern:

```
/*
 * dactyl_manuform.keymap for the Dactyl Manuform 6x6 split keyboard
 */
#include <behaviors.dtsi>
#include <dt-bindings/zmk/keys.h>
#include <dt-bindings/zmk/bt.h>

/ {
    keymap {
        compatible = "zmk,keymap";

        /* — Layer 0: Base (QWERTY) — */
        default_layer {
            bindings = <
                &kp TAB &kp Q &kp W &kp E &kp R &kp T
                &kp LSHFT &kp A &kp S &kp D &kp F &kp G
                &kp LCTRL &kp Z &kp X &kp C &kp V &kp B
                /* ... (remaining rows and thumb cluster) ... */
                &mo 1 &kp SPACE /* thumb: hold=Lower, tap=Space */
            >;
        };

        /* — Layer 1: Lower (Numbers + Navigation) — */
        lower_layer {
            bindings = <
                &trans &kp N1 &kp N2 &kp N3 &kp N4 &kp N5
                &trans &trans &trans &trans &trans &trans
                &trans &trans &trans &trans &trans &trans
                /* ... */
                &trans &trans
            >;
        };

        /* — Layer 2: Raise (Symbols + Function Keys) — */
        raise_layer {
            bindings = <
                &trans &kp F1 &kp F2 &kp F3 &kp F4 &kp F5
                &trans &trans &trans &trans &trans &trans
                &trans &trans &trans &trans &trans &trans
                /* ... */
                &trans &trans
            >;
        };
    };
};
```

7.5.2 ZMK Behaviours: Atomic Key Actions

In ZMK, every key binding is a reference to a 'behaviour', a driver that defines what happens when a key event occurs. This is more flexible than QMK's action system. The most common behaviours used in this keymap are as follows.

The `&kp` (Key Press) behaviour is the simplest: it sends a standard HID keycode when the key is pressed and releases it when the key is released. For example, `&kp Q` sends the HID keycode for the letter 'Q'. The `&mo` (Momentary Layer)

behaviour activates a layer for as long as the key is held. `&mo 1` activates Layer 1 while held; releasing the key immediately deactivates Layer 1 and returns to Layer 0. The `&trans` (Transparent) behaviour is a 'pass-through': it means 'use whatever binding is defined on the next active layer below this one'. This allows lower layers to show through higher ones for keys that have no explicit binding on the active layer.

Advanced behaviours include the `&hold_tap` (Hold-Tap) behaviour, which produces different actions based on whether the key is tapped briefly or held for longer than a threshold (default 200 ms). This is used extensively on the thumb cluster, where a single key can serve as Space on tap and Layer-Momentary on hold, maximising the utility of the limited thumb-cluster key count. The `&bt` (Bluetooth) behaviour controls BLE profile management, allowing the user to switch between multiple paired host devices.

7.5.3 Combos: Multi-Key Simultaneous Triggers

Combos in ZMK allow a specific action to be triggered when two or more keys are pressed simultaneously (within a configurable time window, default 50 ms). They are declared in the keymap file within a `combos` node, separate from the layer bindings. A typical combo declaration looks as follows:

```
/ {
  combos {
    compatible = "zmk,combos";

    /* Pressing Q and W simultaneously sends Escape */
    combo_esc {
      timeout-ms = <50>; /* Both keys must be pressed within 50 ms */
      key-positions = <0 1>; /* Positions 0 and 1 in the binding array */
      bindings = <&kp ESC>;
    };

    /* Pressing both thumb inner keys simultaneously enters bootloader mode */
    combo_boot {
      timeout-ms = <100>;
      key-positions = <32 33>;
      bindings = <&bootloader>;
    };
  };
};
```

Combos are evaluated by the ZMK combo subsystem before key presses are passed to the keymap layer system. When the combo subsystem detects that the `key-positions` specified are all simultaneously active within the `timeout-ms` window, it

fires the `bindings` action and suppresses the individual key events, preventing them from producing their normal layer output.

7.6 Split Keyboard BLE Communication Protocol

The most technically complex aspect of ZMK for a split wireless keyboard is the inter-half communication system. The two halves of the keyboard must synchronise their key matrices in real time over a Bluetooth Low Energy link, while simultaneously the central half maintains a separate BLE connection to the host computer. This requires the central half to act as both a BLE Central (in the inter-half link) and a BLE Peripheral (in the host link) a sophisticated role inversion that is only possible due to the nRF52840's multi-role BLE capability.

7.6.1 BLE Central and Peripheral Roles

In Bluetooth Low Energy terminology, a Central device initiates connections and scans for devices, while a Peripheral device advertises itself and accepts connections. In ZMK's split architecture, the right half (the master, or central half) acts as the BLE Central with respect to the left half, meaning it initiates and manages the inter-half BLE connection. The left half (the peripheral, or slave half) acts as the BLE Peripheral in the inter-half link, advertising its presence and accepting the connection from the central half. Additionally, the central half simultaneously acts as a BLE Peripheral to the host computer advertising as a standard BLE HID keyboard and accepting a connection from the host. The peripheral half has no direct connection to the host.

Parameter	Central Half (Right)	Peripheral Half (Left)
ZMK Role	Central + HID Peripheral	Split Peripheral only
Host Connection	Yes (advertises as HID keyboard)	No
Inter-half Role	BLE Central (initiates link)	BLE Peripheral (advertises)
Battery Reporting	Yes (to host)	No
Firmware File	dactyl_manuform_right-...uf2	dactyl_manuform_left-...uf2

Table 7.2: BLE Role Assignment for Each Keyboard Half

7.6.2 The 7.5 ms Connection Interval

BLE communication is not continuous. It is organised into connection events that occur at a fixed interval called the Connection Interval. Between connection events, both devices power down their radio transceivers to conserve energy. The Connection Interval is negotiated between the Central and Peripheral during connection establishment. ZMK configures the inter-half connection interval to its minimum permissible BLE value: 7.5 milliseconds. This means the peripheral half can deliver a new key state update to the central half at most every 7.5 ms. A keypress that occurs immediately after a connection event will be reported to the central at the next connection event, at most 7.5 ms later. This latency is imperceptible to the human nervous system; typical human perception of input lag only becomes noticeable above approximately 30–50 ms.

The total end-to-end latency from keypress on the peripheral half to character appearing on screen is the sum of several components: the matrix scan period (approximately 1 ms), the debounce confirmation window (5 ms), the inter-half BLE transmission (up to 7.5 ms), and the host BLE connection interval (typically 7.5–15 ms). The theoretical maximum total latency is therefore approximately 29 ms, which is comfortably within the imperceptible threshold.

7.6.3 ZMK Split Transport Data Format

The peripheral half does not transmit individual key events over the inter-half BLE link. Instead, it transmits a compact binary representation of the complete state of its 6x6 matrix 36 bits, rounded to 5 bytes at each connection event. This state-based approach (as opposed to event-based) ensures that if a BLE connection event is missed due to radio interference, the central will receive the correct, up-to-date state at the next event, rather than missing an event message permanently. The central half merges the received peripheral state with its own local matrix state and feeds the combined 66-key state to the ZMK keymap processing engine, which resolves layers, combos, and behaviours before generating HID reports for the host.

7.6.4 BLE Bonding and Security

ZMK uses BLE bonding (long-term key pairing) for all connections. When the central half first pairs with the host computer, a cryptographic key exchange occurs and the keys are stored in the nRF52840's non-volatile storage. Subsequent connections are encrypted automatically without requiring re-pairing. ZMK supports up to five BLE host profiles, allowing the keyboard to remain bonded to five different host devices simultaneously and switch between them using the `&bt BT\_SEL` key binding. The inter-half bond is established once during initial setup and is maintained persistently.

8. SYSTEM WORKING AND VALIDATION

This chapter provides a detailed technical examination of the three principal firmware-controlled operational aspects of the keyboard signal debouncing, power state management, and system-level validation through structured testing. Together, these elements ensure the keyboard delivers reliable, ergonomically sound, and energy-efficient performance during sustained daily use.

8.1 Signal Debouncing Algorithm

Mechanical key switches exhibit a well-known phenomenon called contact bounce. When the gold crosspoint contacts of the Outemu Brown switch make or break, the spring-loaded mechanical contact does not transition cleanly from open to closed (or vice versa) in a single step. Instead, the contact bounces rapidly, opening and closing multiple times over a period of approximately 1–5 milliseconds before settling into its final state. If uncorrected, the matrix scanner would interpret each bounce event as a separate keypress, resulting in a single physical keystroke registering as two, three, or more characters.

8.1.1 The Eager Debounce Algorithm

ZMK implements the 'eager' debounce algorithm by default, which is configured with a debounce period of 5 milliseconds (``CONFIG_ZMK_KSCAN_DEBOUNCE_PRESS_MS=5`` and ``CONFIG_ZMK_KSCAN_DEBOUNCE_RELEASE_MS=5``). The algorithm operates as follows:

1. When the matrix scanner first detects a change in a key's state (e.g., a column pin transitions from LOW to HIGH during a row scan), it immediately registers a provisional key event. This is the 'eager' aspect: the event is reported at the first detected edge, minimising perceived latency.
2. Simultaneously, a 5 ms debounce timer is started for that key position.
3. During the 5 ms window, any further transitions on that key's position (the bouncing contacts) are recorded but do not generate new key events.

4. At the end of the 5 ms window, the key's state is sampled. If it matches the initially reported state, the event is confirmed. If the state has changed back (a very brief 'phantom' press), the event is cancelled.
5. Any new key transition after the debounce window has elapsed restarts the process.

Mathematically, for a key press at time $t=0$, the debounce function $D(t)$ can be expressed as: a key event E is confirmed if and only if the key state $S(t)$ at $t = 5$ ms equals the state $S(0)$ detected at $t = 0$. This ensures that only genuine, sustained contact closures generate key events. The 5 ms window was chosen as it exceeds the bounce duration of virtually all mechanical switches tested (the Outemu Brown typically bounces for 1–2 ms) while remaining far below the minimum inter-keystroke interval of a fast typist (approximately 50 ms at 200 WPM).

8.2 Power Management and Sleep State Machine

Battery-powered wireless devices must aggressively manage power consumption to achieve practical runtimes. The nRF52840 SoC supports several distinct power modes, and ZMK implements a state machine that transitions through these modes based on user activity. The three primary power states are Active Mode, Idle Mode, and Deep Sleep Mode.

8.2.1 Active Mode

In Active Mode, the nRF52840 CPU operates at its full 64 MHz clock frequency, the matrix scanner polls the GPIO matrix every 1 ms, and the BLE radio is active at the 7.5 ms connection interval. This mode delivers maximum responsiveness but also maximum current consumption. Based on Nordic Semiconductor characterisation data, the nRF52840 draws approximately 1.5–3.0 mA in this mode (depending on BLE advertising and connection activity).

8.2.2 Idle Mode

After a configurable period of inactivity (default 30 seconds of no keypresses), ZMK transitions to Idle Mode. In this state, the matrix scan rate is reduced (to approximately 100 ms intervals), and the BLE connection interval is renegotiated to a longer value (e.g., 50–500 ms), reducing radio duty cycle. The CPU enters a low-power wait state between scan events, using the Zephyr tick-less idle kernel mode. Current consumption in Idle Mode drops to approximately 0.3–0.8 mA. Any keypress

immediately triggers a GPIO interrupt that wakes the CPU and restores Active Mode within one matrix scan cycle.

8.2.3 Deep Sleep Mode

After a longer period of inactivity (default 15 minutes, configurable via `CONFIG_ZMK_SLEEP_IDLE_TIMEOUT`), ZMK initiates Deep Sleep. In this state, the CPU clock is stopped, all peripherals are powered down, and the BLE connection to the host is terminated. Only the GPIO interrupt controller remains powered, consuming approximately 2–5 microamperes (0.002–0.005 mA). Upon a keypress, the GPIO interrupt triggers a full system wake-up: the CPU restarts, ZMK re-initialises the BLE stack, and re-establishes the connection to the host. This wake-up process takes approximately 1–3 seconds, which is the only perceptible latency associated with the power management system.`

8.2.4 Power Consumption Analysis and Battery Life Estimation

To estimate battery longevity, a usage model is defined: 8 hours of daily use, comprising 2 hours of active typing (Active Mode), 4 hours of computer-on but keyboard-idle (Idle Mode), and 16 hours of keyboard powered off or in Deep Sleep. Using the current consumption values above, the estimated average current draw per half over a 24-hour period is calculated as follows:

```
Average daily current (per half):
Active:  2 hr x 2.5 mA = 5.00 mAh
Idle:    4 hr x 0.5 mA = 2.00 mAh
Deep Sleep: 18 hr x 0.003 mA = 0.05 mAh
Daily Total:           = 7.05 mAh

Battery Capacity: 500 mAh
Estimated runtime: 500 / 7.05 = ~70.9 days per charge
```

This analysis yields an estimated battery life of approximately 10 weeks per charge under the defined usage model. Empirical testing over a 4-week observation period confirmed that each half retained sufficient charge for the full duration without recharging, consistent with the theoretical estimate. This figure significantly exceeds the battery life of comparable commercial wireless keyboards (typically 1–4 weeks), owing to the deep sleep capability of the nRF52840 and ZMK's aggressive power management.

8.3 Testing and Results

Systematic testing was conducted across three categories: BLE pairing reliability, typing ergonomics evaluation, and battery longevity validation. All tests were conducted over a period of four weeks of daily use by the project author.

8.3.1 BLE Pairing and Connection Reliability

BLE pairing was tested across three host operating systems: Windows 11, Ubuntu 22.04 LTS (Linux), and Android 13. On Windows 11, initial pairing completed in under 10 seconds on all three test attempts, and reconnection upon wake-from-sleep was consistently achieved within 2 seconds. On Ubuntu 22.04 with a Bluetooth adapter using the Intel AX200 controller, connection instability was initially observed. This was resolved by adding ``CONFIG_ZMK_BLE_EXPERIMENTAL_CONN=y`` and ``CONFIG_ZMK_BLE_EXPERIMENTAL_SEC=y`` to the firmware configuration, a known workaround documented in the ZMK community for certain Linux Bluetooth controllers. After this fix, connection reliability on Linux was equivalent to Windows.

Test Parameter	Result	Pass/Fail
Initial BLE pairing (Windows 11)	< 10 seconds	Pass
Reconnection after deep sleep (Windows 11)	< 2 seconds	Pass
Initial BLE pairing (Ubuntu 22.04)	< 15 sec (after BLE fix)	Pass
Inter-half link establishment	< 3 seconds after power-on	Pass
Key dropout at 5 m range	Zero dropouts over 30 min	Pass
Key dropout at 10 m range (LOS)	< 0.1% dropout rate	Pass
Multi-key rollover (6 simultaneous keys)	All 6 keys registered	Pass

Table 8.1: BLE Pairing and Connectivity Test Results

8.3.2 Key Registration Accuracy

Key registration accuracy was evaluated using a typing test suite (typing.com advanced test), recording 500 words across 5 test sessions. No key ghosting events (unintended key registrations due to matrix cross-coupling) were observed in any test

session, confirming the correct functioning of the 1N4148 diode anti-ghosting circuit. No double-registrations attributable to inadequate debouncing were recorded, validating the 5 ms debounce window for the Outemu Brown switches used.

8.3.3 Typing Ergonomics Evaluation

A qualitative ergonomics evaluation was conducted by the project author, who used the keyboard as the exclusive input device for all computer work over a 4-week period. The following subjective observations were recorded. After an initial adaptation period of approximately 5 days (during which typing speed temporarily decreased from the baseline), the concave key-well geometry was reported to significantly reduce the perceived effort per keystroke, as fingers did not need to extend fully to reach keys at the extremities of each row. The split layout allowed the shoulders to remain at a natural width throughout typing sessions, eliminating the shoulder adduction required by standard keyboards. No wrist pain was reported during the evaluation period, compared to intermittent wrist discomfort experienced with a standard flat keyboard. Typing speed recovered to baseline (approximately 85 WPM) within 10 days of adaptation.

8.3.4 Battery Longevity Test

Both halves were fully charged and then used continuously under the defined usage model (described in Section 8.2.4) without recharging for 28 days. At the end of the 28-day period, the right half (central) reported an estimated 58% battery remaining, and the left half (peripheral) reported 62% remaining (battery level is estimated by the nRF52840 via ADC voltage measurement of the battery terminal and reported via ZMK's battery service). This empirical result exceeds the minimum 4-week practical runtime threshold and is consistent with the theoretical estimate of approximately 10 weeks per charge.

9. CONCLUSION

This project successfully demonstrates the design and implementation of a split ergonomic concave keyboard aimed at reducing strain associated with prolonged typing. The system combines mechanical design, embedded firmware, and wireless communication into a cohesive and functional solution.

The hardware integrates a BLE-enabled microcontroller based on the nRF52840, a 6×6 key matrix with diode isolation to support full N-key rollover, and a 3D-printed concave enclosure developed using ergonomic considerations. The split layout and curved key-well geometry promote a more natural hand position, reducing wrist deviation and improving typing comfort over extended periods.

On the firmware side, the system is built using the ZMK framework on top of the Zephyr RTOS, enabling efficient task scheduling, reliable matrix scanning, and low-latency Bluetooth communication. The split architecture allows one half to function as the central unit while the other operates as a peripheral, ensuring seamless coordination between both halves. The use of a cloud-based CI/CD pipeline simplifies firmware deployment, while the configurable keymap and layered behavior system provide flexibility for user-specific customization.

System validation confirms that the keyboard meets all core functional requirements, including accurate key detection, stable BLE connectivity across multiple devices, and effective power management for extended battery life. The implemented design also demonstrates consistent performance under continuous usage conditions.

Overall, the project delivers a complete and technically sound solution that leverages open-source tools and modern embedded systems design principles. It highlights the feasibility of developing customizable ergonomic input devices that address real-world usability and health concerns while maintaining performance and scalability.

10. FUTURE SCOPE

The current implementation provides a solid foundation for further enhancements in both hardware integration and user experience.

A primary improvement would be the transition from a hand-wired matrix to a custom-designed PCB. Using tools such as KiCad, a dedicated PCB potentially incorporating a flexible or multi-layer design would improve structural reliability, reduce assembly complexity, and enable a more compact and refined enclosure. Integration of surface-mount components would further optimize space and durability.

Additional hardware features supported by ZMK can be incorporated to enhance usability. For instance, OLED displays can be integrated using ZMK's display subsystem (`CONFIG_ZMK_DISPLAY= y`) to provide real-time information such as active layers, BLE connection status, and battery levels. This would significantly improve user interaction, especially in multi-device environments.

Haptic feedback can also be explored through the integration of vibration motors controlled via GPIO pins. With appropriate firmware support, such feedback mechanisms can be linked to key events or system states, offering silent tactile confirmation of inputs.

Further enhancements may include improved power management through optimized firmware configurations and hardware components, extending battery life for prolonged usage.

Finally, comprehensive ergonomic validation through user studies involving diverse hand sizes and typing behaviors would provide quantitative data to better assess reductions in RSI-related strain compared to conventional keyboard designs.

REFERENCES

- [1] ZMK Firmware Project, "ZMK Documentation," zmk.dev, 2024. [Online]. Available: <https://zmk.dev/docs>
- [2]] Zephyr Project, "Pro Micro nRF52840 Board Documentation," 2024. [Online]. Available: https://docs.zephyrproject.org/latest/boards/others/promicro_nrf52840/doc/index.html
- [3] Zephyr Project, "Zephyr RTOS Documentation," docs.zephyrproject.org, 2024. [Online]. Available: <https://docs.zephyrproject.org>
- [4] TurkeyBoards, "QMK and ZMK Firmware Options," 2024. [Online]. Available: <https://docs.turkeyboards.com/turkeyboards-docs/buyers-guide/qmk-and-zmk-firmware-options>
- [5] A. C. Antona, C. Stephanidis, and J. Clarkson, "Ergonomic Evaluation of Alternative Keyboard Designs," *Proc. Int. Conf. Engineering and Product Design Education*, 2015. [Online]. Available: https://researchportal.northumbria.ac.uk/ws/portalfiles/portal/3037317/Ergo_keyboards.pdf
- [6] StatPearls Publishing, "Carpal Tunnel Syndrome," StatPearls, 2024. [Online]. Available: <https://www.ncbi.nlm.nih.gov/books/NBK448179/>
- [7] O. K. Asikhia and S. E. Ehondor, "Ergonomics Design of Computer Keyboard Layout," *J. Sci. Multidiscip. Res.*, vol. 2, pp. 34–39, 2010. [Online]. Available: <https://www.cenresinjournal.com/wp-content/uploads/2020/03/page-34-39004.pdf>
- [8] Masterzen, "Designing a Keyboard from Scratch," 2020. [Online]. Available: <https://www.masterzen.fr/2020/05/03/designing-a-keyboard-part-1/>
- [9] N. H. Baba and D. D. I. Daruis, "Repetitive Strain Injury (RSI) among Computer Users: A Case Study in Telecommunication Company," *Malaysian Journal of Public Health Medicine*, pp. 48–52, 2016. [Online]. Available: https://www.researchgate.net/publication/317219877_Repetitive_Strain_Injury_RSI_among_computer_users_A_case_study_in_telecommunication_company
- [10] splitkb, "Halcyon Split Keyboard Repository," 2025. [Online]. Available: <https://github.com/splitkb/halcyon>
- [11] FreeCAD Association, "FreeCAD 0.21 User Manual," freecad.org, 2024.
- [12] Outemu (Gaote), "Outemu Brown Mechanical Switch Datasheet," Shenzhen Gaote Electronics Co., Ltd., 2022.
- [13] Gateron, "Gateron Hot-Swap Socket MX Specification," Gateron Group, Huizhou, China, 2023.
- [14] The complete firmware configuration and keymap implementation are available in the project repository : <https://github.com/tuxy-streamer/zmk-config>